

DOCUMENTAZIONE

SISTEMA DI GESTIONE TIROCINI E SEDUTE DI LAUREA

INTRODUZIONE

Link repository github: <https://github.com/OOP2026/Gruppo01>

Federico Merola (DE1000291) - Pasquale Mencocco(DE1000430)

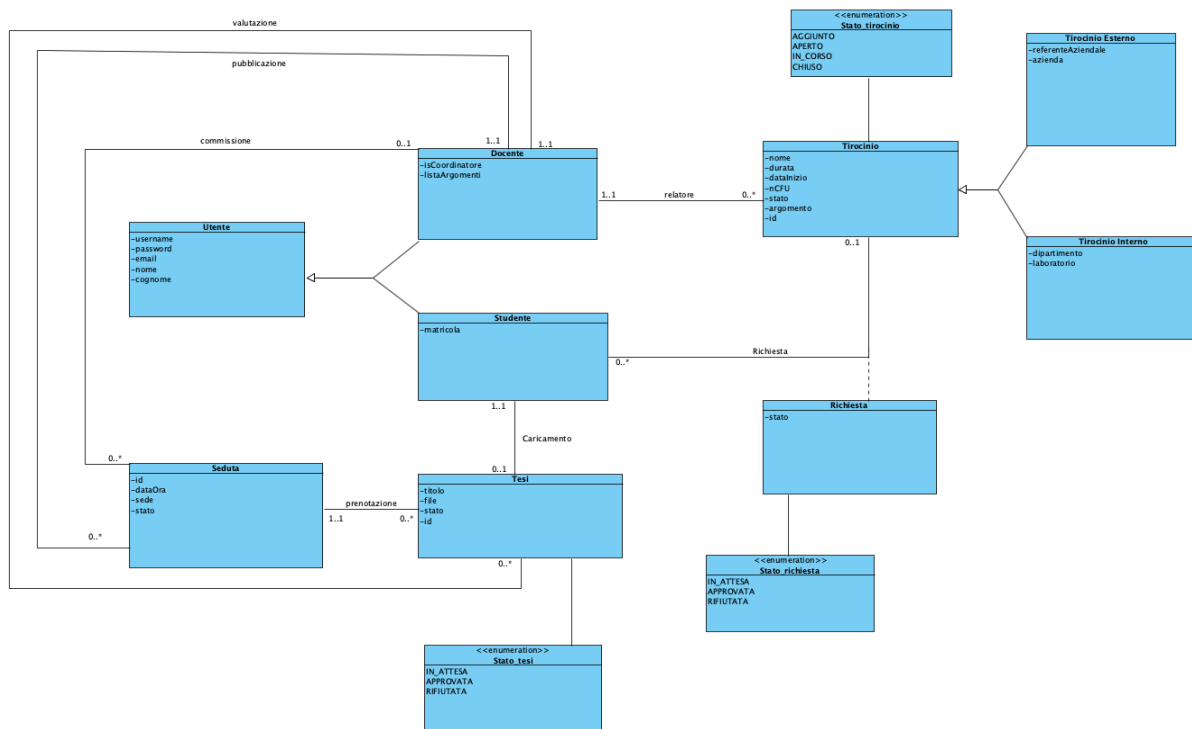
Il presente documento descrive l'architettura di un software che gestisce l'iscrizione ai tirocini e alle sedute di laurea di un'Università. Si tratta, in particolare, di un applicativo Java dotato di interfaccia grafica (GUI) in Swing e supportato da una base di dati relazionale (implementata con PostgreSQL), in grado di gestire:

- registrazione e login di studenti e docenti alla piattaforma;
- lato studente, la prenotazione ai tirocini, il caricamento della tesi e la conseguente iscrizione alla seduta di laurea;
- lato docente, la valutazione delle richieste di partecipazione ad un tirocinio, la valutazione delle tesi caricate dallo studente e, nel caso in cui il docente sia il coordinatore del corso di laurea, la gestione delle commissioni e della seduta di laurea.

L'applicazione è strutturata secondo il pattern architetturale BCE+DAO, per garantire una separazione tra la logica di dominio, il flusso applicativo e l'interfaccia utente.

Si specifica che nonostante il gruppo sia formato da due persone, lo sviluppo comprende la parte della traccia destinata ad un gruppo di 3 componenti (per problemi interni in fase di sviluppo del progetto).

PARTE 1: DIZIONARIO PACKAGE MODEL UML



DIZIONARIO CLASSI

- Utente (nome, cognome, email, username, password) {totale, disgiunta}:
 - docente (isCoordinatore, listaArgomenti)
 - studente (matricola)
- Tirocinio (id, nome, durata, dataInizio, nCFU, stato, argomento) {totale, disgiunta}:
 - tirocinio esterno (azienda, referenteAziendale)
 - tirocinio interno (dipartimento, laboratorio)
- Richiesta (stato)
- Tesi (id, titolo, file, stato)
- Seduta (id, dataOra, sede, stato)
- Stato_Richiesta (IN_ATTESA, APPROVATA, RIFIUTATA)
- Stato_Tirocinio (AGGIUNTO, APERTO, IN_CORSO, CHIUSO)
- Stato_Tesi (IN_ATTESA, APPROVATA, RIFIUTATA)

UTENTE : classe generica per identificare le persone nel minimondo. Un utente può essere o uno studente o un docente (per questo la generalizzazione è disgiunta)

STUDENTE: lo studente differisce dall'utente generico per il numero di matricola identificativo.

DOCENTE: isCoordinatore è un attributo di tipo booleano che indica se il docente in questione è o meno il coordinatore. Nel caso in cui il docente sia coordinatore avrà permessi speciali gestiti tramite java.

TIROCINIO: con argomenti tirocinio si indica una stringa contenente le informazioni inerenti al singolo tirocinio seguendo i dettami della traccia, secondo la quale ogni docente inserisce oltre alla struttura (nome, durata, e inizio), “gli argomenti del tirocinio”. Dunque ogni tirocinio ha un suo argomento specifico, mentre al docente è associata una lista di argomenti. Per ogni tirocinio vengono indicati anche il numero di CFU e i posti disponibili.

TIROCINIO_ESTERNO: Si distingue, come da traccia, solo per il nome del referente aziendale (esterno al sistema, dunque una semplice stringa), oltre che al nome dell'azienda stessa ospitante.

TIROCINIO_INTERNO: Si distingue per il dipartimento di riferimento e per il dipartimento ed il laboratorio in cui si svolge.

RICHIESTA: Classe relazionale, l'attributo stato può assumere, come vedremo, i 3 valori definiti da una classe di tipo *enumerazione*. Se viene rifiutata, l'istanza Richiesta non viene eliminata subito dal sistema, ma viene settata momentaneamente a RIFIUTATA: quando lo studente proverà a inserire una nuova richiesta, quella vecchia verrà eliminata.

TESI: Il sistema ne memorizza il titolo, e il docente incaricato di valutarla può modificarne l'attributo “stato”: se viene rifiutata l'istanza Tesi non viene eliminata subito dal sistema, ma viene settata momentaneamente a RIFIUTATA: quando lo studente proverà a inserire una nuova tesi, quella vecchia verrà eliminata. L'istanza comprende anche la seduta a cui lo studente vuole partecipare.

SEDUTA: definita da sede e dataOra, l'attributo stato è un booleano che indica se è aperta (disponibile a prenotazioni) o chiusa (per iniziativa del coordinatore, colui che crea la seduta e la chiude).

STATO: Le Classi *enumerazione* che contengono i 3 (o 4, nel caso del tirocinio) valori che può assumere l'attributo stato in relazione a Richiesta, Tirocinio e Tesi.

DIZIONARIO ASSOCIAZIONI

Studente (1..1) - CARICAMENTO - (0..1) Tesi: Uno studente può caricare una sola tesi alla volta, mentre una tesi sarà sempre quella caricata dal singolo studente.

Docente (1..1) - RELATORE - (1..*) Tirocinio: Ogni tirocinio ha per relatore il singolo Docente, mentre il docente può avere più argomenti di tirocinio, e dunque essere relatore di più tirocini.

Studente (0..*) RICHIESTA (0..1) Tirocinio: Lo studente, come da traccia, può fare richiesta di un solo tirocinio alla volta, mentre il tirocinio può essere richiesto da un certo numero di studenti.

Docente (1..1) VALUTAZIONE (0..*) Tesi: la tesi viene sempre valutata dal singolo docente, mentre un docente ha la facoltà di valutare più tesi.

Docente (1..1) PUBBLICAZIONE (0..*) Seduta: il vincolo per tale relazione è che il docente in questione sia il coordinatore del corso; solo lui può pubblicare le sedute senza un limite massimo.

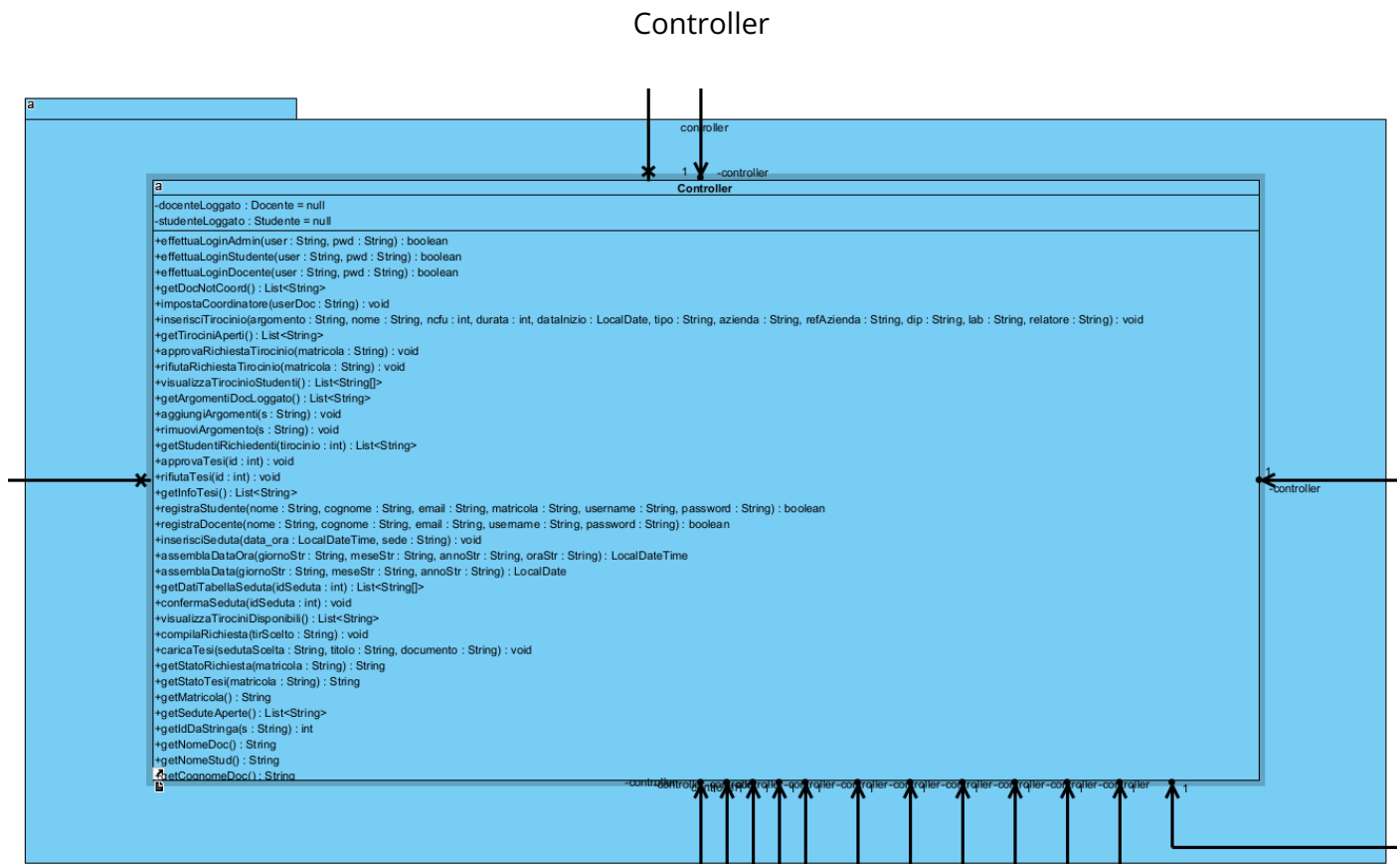
Tesi (0..*) PRENOTAZIONE (1..1) Seduta: L'associazione tra tesi e seduta nasce nel momento stesso in cui viene caricata la tesi, dunque ogni tesi è sempre associata ad una e una sola seduta (quella per cui prova a prenotarsi lo studente), mentre ad una seduta corrispondono n tesi.

Docente (0..*) COMMISSIONE (0..*) Seduta : Alla seduta partecipano un determinato numero di docenti, quelli che ne compongono la commissione.

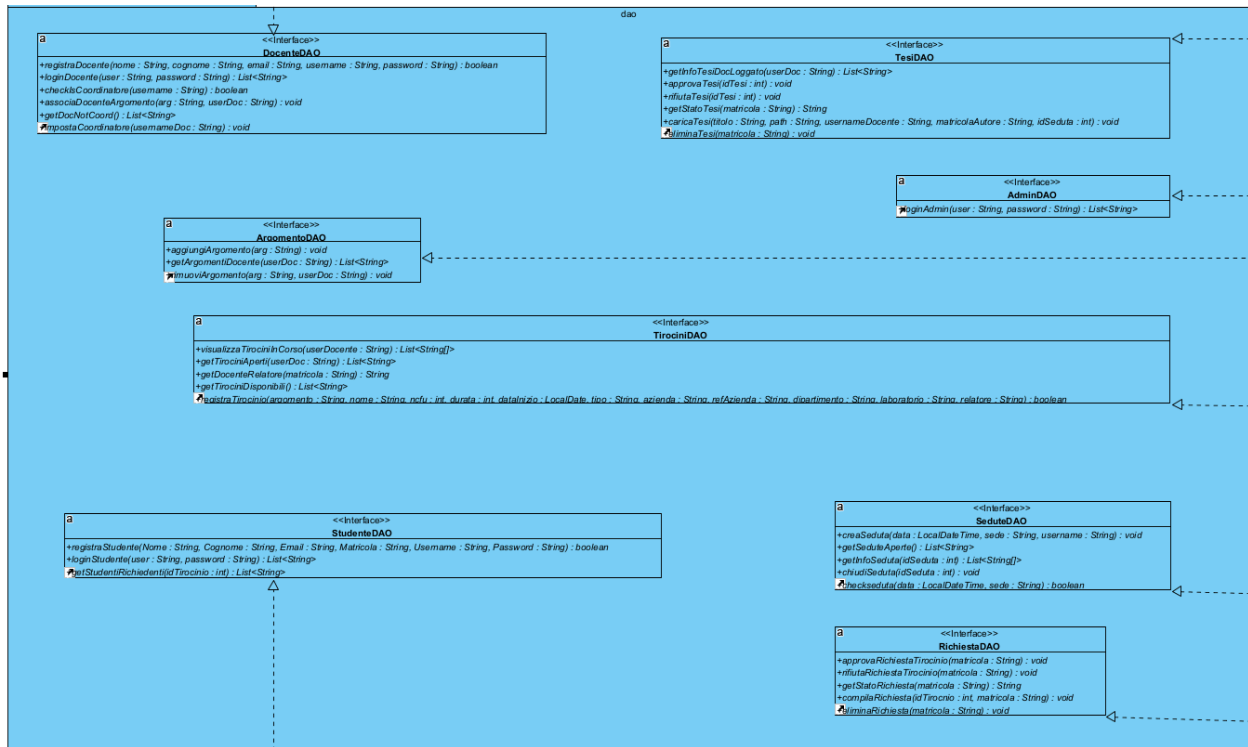
ND: In aggiunta, è stata creata una classe speciale Admin, che ha la facoltà di inserire nel sistema i tirocini e rendere coordinatore un docente tramite apposite GUI.

PARTE 2: PROGETTAZIONE DELLA SOLUZIONE

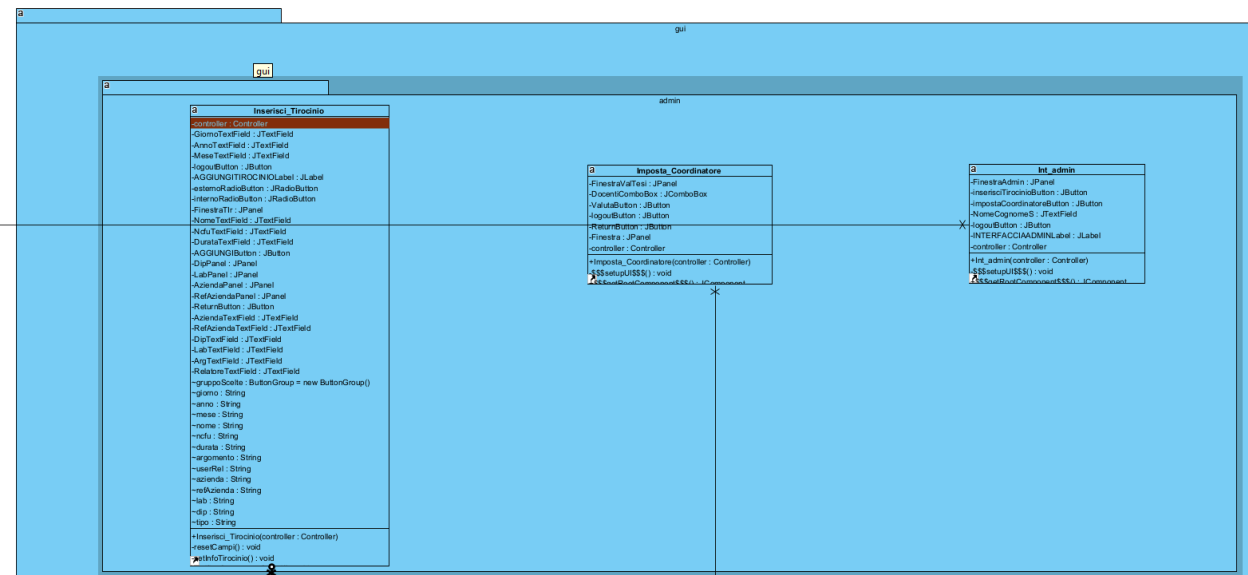
Per una visualizzazione completa del diagramma, si rimanda al file in formato .vpp presente nella cartella documentazioneFinale.



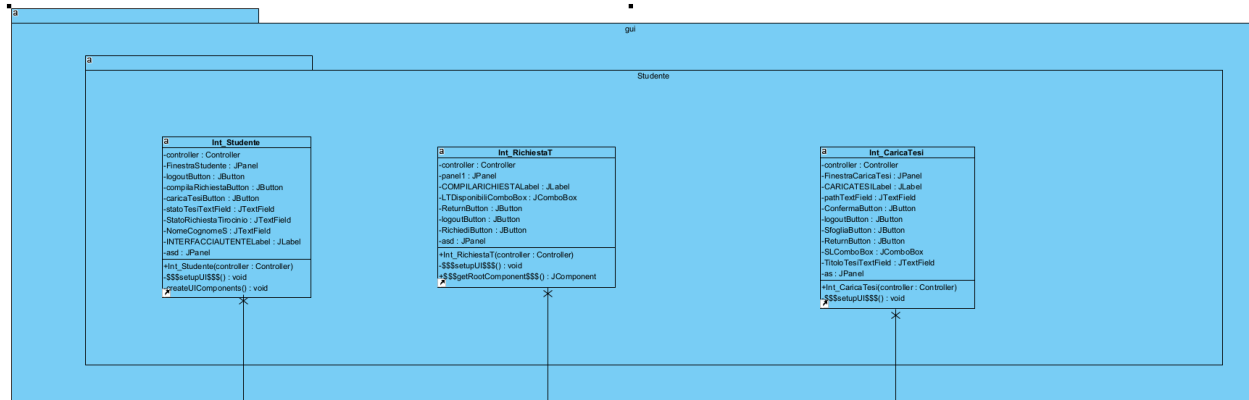
DAO



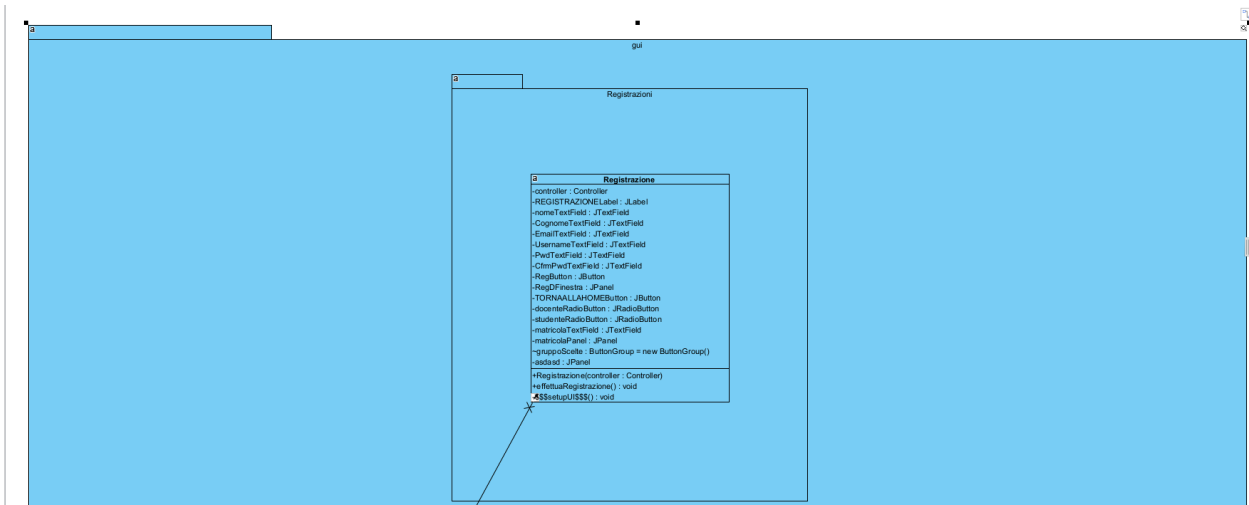
GUI- Admin



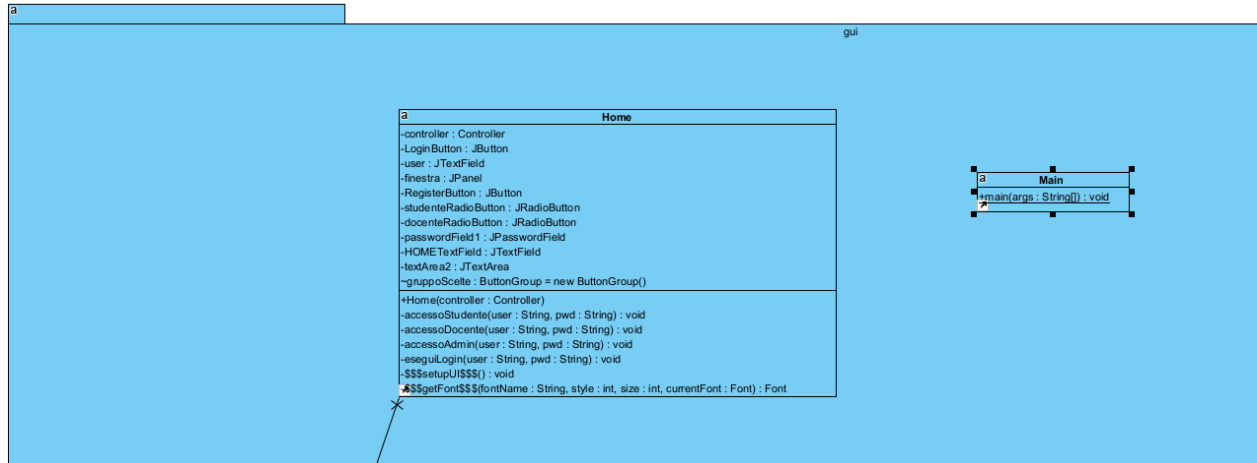
GUI-Studente



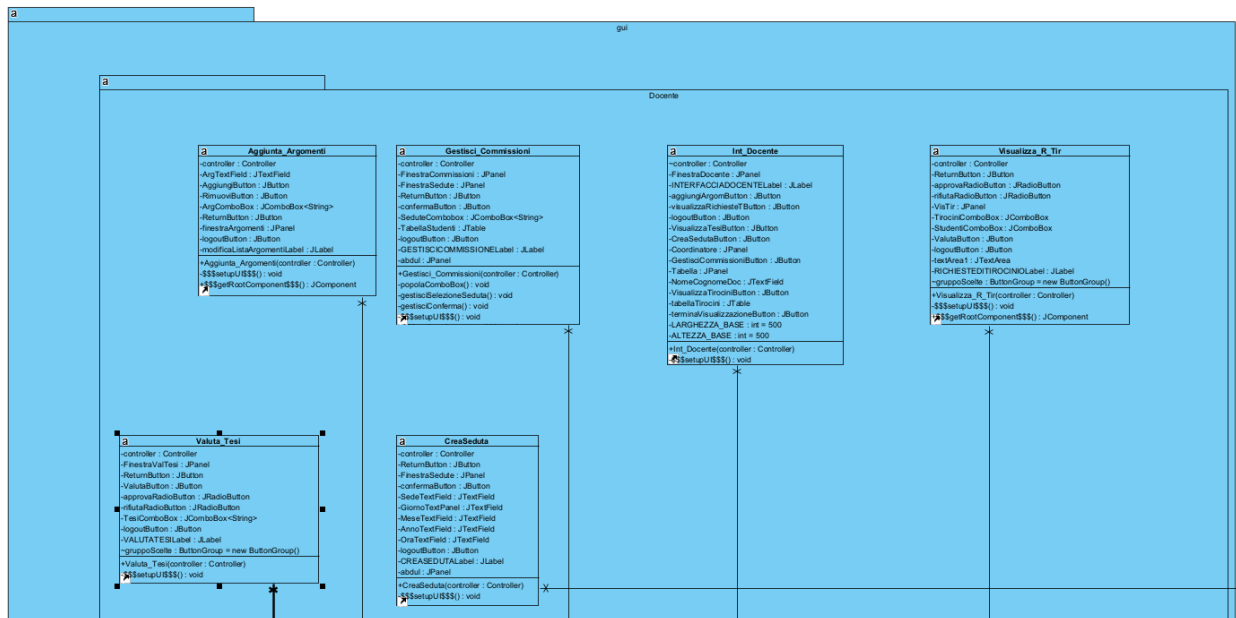
GUI - REGISTRAZIONE



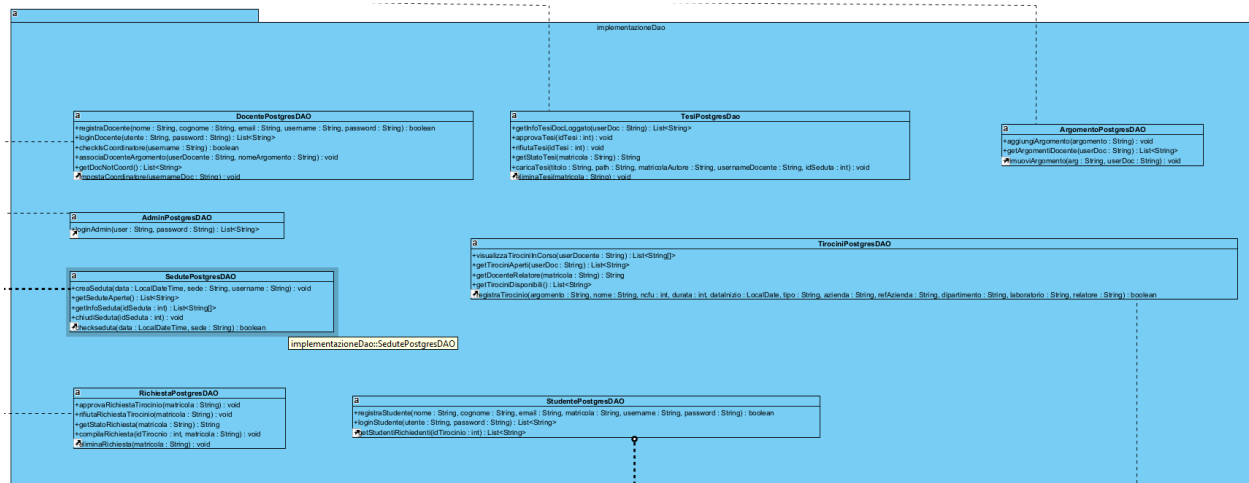
Gui Home



GUI Docente



Implementazione DAO



PARTE 3: ARCHITETTURA DEGLI ALTRI PACKAGE

Oltre al Model Package, gli altri 3 package fondamentali (escludendo ConnessioneDatabase, il cui compito è stabilire una connessione con il Database) sono quelli del controller, della GUI e del DAO.

CONTROLLER

Il livello di Controllo è stato implementato tramite un'unica classe Controller. Il suo scopo principale è ricevere gli input generati dalle GUI, interrogare o modificare di conseguenza il Database, e restituire i dati elaborati all'interfaccia, garantendo il totale isolamento del livello di Dominio dalla logica di presentazione.

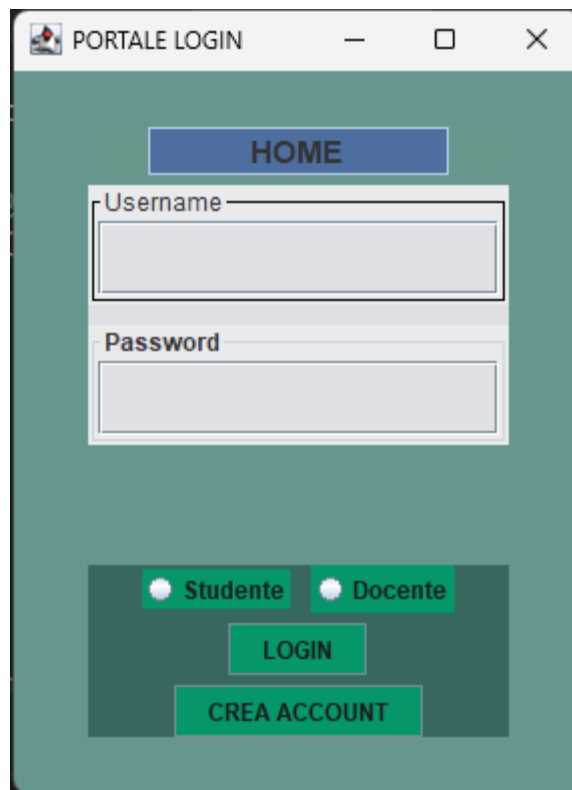
Nel progettare il Controller, sono state adottate le seguenti scelte architetturali:

- Nessuna classe della GUI contiene codice operativo. Ogni azione complessa (es. l'estrazione degli studenti in attesa di approvazione per un tirocinio, il salvataggio dei percorsi dei file tesi o la verifica delle credenziali al momento del login) è delegata a metodi specifici esposti dal Controller. La GUI si limita a invocare il metodo passando i parametri necessari e ad aggiornare la vista in base al valore di ritorno.
- Tracciamento degli Utenti Attivi: Il Controller mantiene il riferimento logico all'utente attualmente loggato (Studente, Docente) tramite un attributo interno. Questo meccanismo permette, ad esempio, al metodo di associare i tirocini da valutare al docente attualmente loggato tramite la sua primary key "username".
- Gestione delle Eccezioni: Il Controller è responsabile della sanitizzazione dei dati e del controllo dei vincoli logici prima di interrogare il Database. In caso di errore logico o di sistema, il Controller intercetta l'anomalia e restituisce messaggi di errore standardizzati alla GUI, che provvederà unicamente a mostrarli tramite appositi avvisi a schermo.

INTERFACCE E MANUALE D'USO

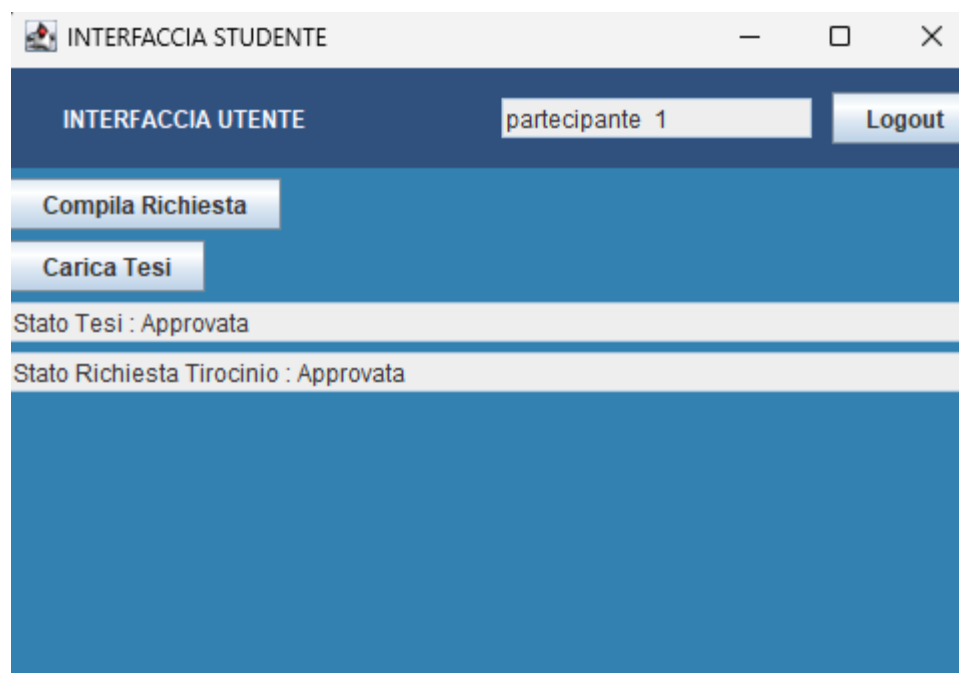
Home

Al momento dell'avvio del programma, a comparire è una schermata Home da cui è possibile sia effettuare il Login che Registrarsi. Nel secondo caso, si aprirà la schermata di Registrazione dove l'utente potrà compilare i campi necessari, specificando il proprio ruolo, e, nel caso dello studente, indicando l'apposita matricola. Nel caso si prema invece il pulsante di Login, si apriranno le schermate del Docente o dello Studente che ha effettuato il Login (o dell'admin, nel caso in cui non abbia scelto né Docente né Studente).



Interfacce Studente

Una volta effettuato il login, lo Studente vede a schermo lo stato della richiesta di Tirocinio e lo stato della Tesi. Inoltre premendo sugli appositi JButton si apriranno due dashboard, una per il caricamento della Tesi (lo Studente la carica scegliendo il file dal suo dispositivo) e la dashboard per effettuare la richiesta di partecipazione al tirocinio. Per entrambe, è possibile sia tornare alla schermata principale che effettuare il logout e tornare direttamente alla Home.



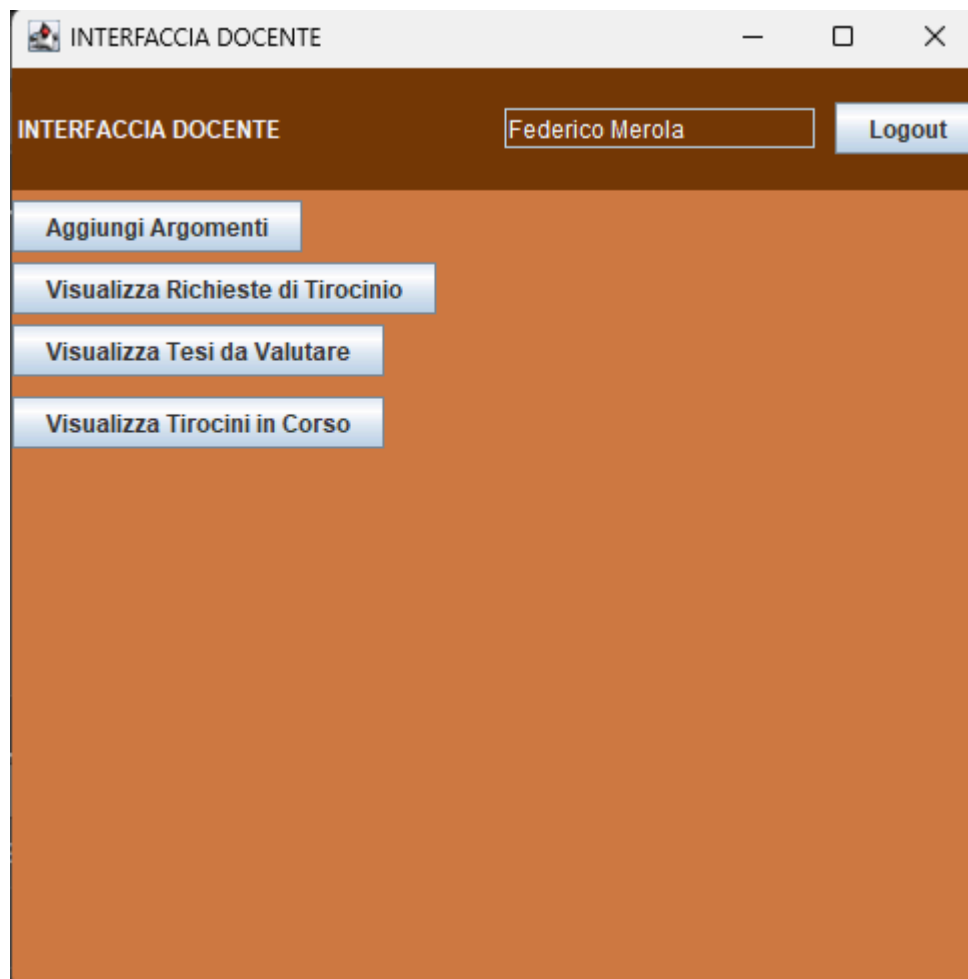
Interfacce Docente

La schermata principale del Docente funge da crocevia (tramite gli appositi JButton) per tutte le singole dashboard che permetteranno al docente di:

- Aggiungere (e rimuovere) gli argomenti associati a lui e ai tirocini di cui sarà amministratore;
- Valutare le richieste di partecipazione degli Studenti ai tirocini di cui è amministratore (scegliendo semplicemente tra "approva" e "rifiuta");

-
- Valutare le Tesi degli Studenti(scegliendo semplicemente tra “approva” e “rifiuta”);
 - Visualizzare i Tirocini in Corso da lui gestiti.

Inoltre, se ad accedere è stato il Coordinatore del corso, risulteranno visibili due opzioni in più: “Crea Nuova Seduta”, da cui si passa alla dashboard in cui viene inizializzato l’oggetto Seduta, e “Gestisci Commissioni”, da cui si passa alla dashboard in cui viene approvata e chiusa la seduta con la relativa commissione (il filtraggio dei docenti che hanno approvato almeno una Tesi è gestito automaticamente dal controller).



DAO

Il compito del DAO è quello di fare da intermediario tra il controller e il database in PostgreSQL. Ad esso sono affidate l'esecuzione delle query per recuperare i dati necessari al corretto/coerente funzionamento del programma.

```
public boolean checkIsCoordinatore(String username){ 1 usage  👤 kage-012
    String sql = "SELECT iscoordinatore FROM DOCENTE WHERE Username = ?";
    try (Connection conn = ConnessioneDatabase.getInstance();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString( parameterIndex: 1, username);

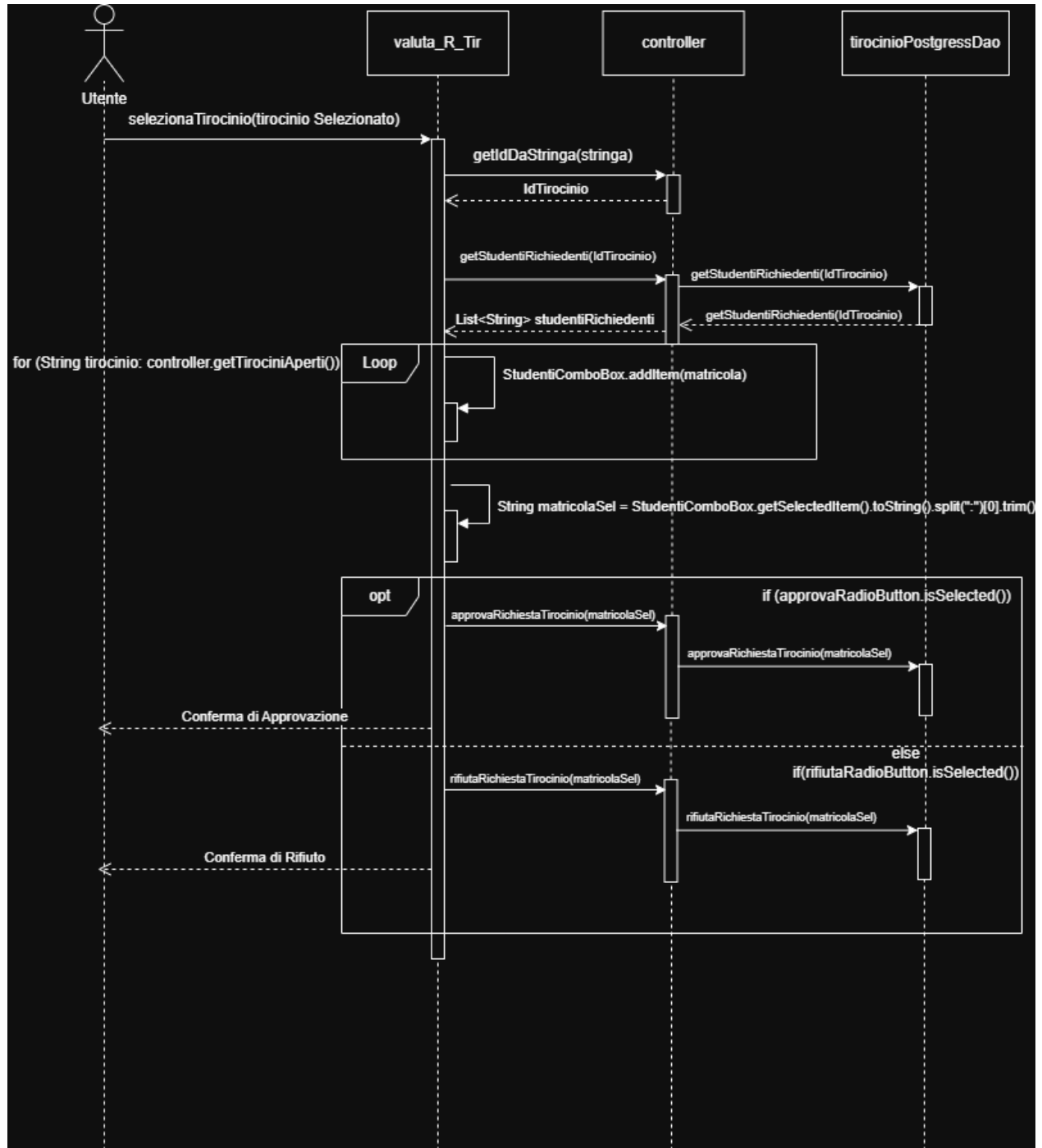
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                return rs.getBoolean( columnLabel: "iscoordinatore");
            }
        }

    } catch (SQLException e) {
        System.err.println("Errore SQL durante il login dello studente: " + e.getMessage());
    }

    return false;
}
```

PARTE 4: SEQUENCE DIAGRAM

Valuta Richieste di Tirocinio



Assembla DataOra

